

LAB : Agents avec Langchain**Table des matières**

Création d'un nouveau agent sans System Message	2
Agent avec System Message : un message de contrôle qui définit le comportement global du modèle.....	2
Agent avec Few-shot learning : une méthode où le modèle apprend une nouvelle tâche ou classe à partir de quelques exemples seulement.....	3
Agent avec réponse structurée: une sortie organisée selon un format prédéfini, plutôt que du texte libre.....	3
Agent avec réponse structurée en utilisant BaseModel : rendre la réponse facile à exploiter automatiquement par un programme ou un système.	4
Les tools (outils) : une fonction externe que le modèle peut appeler pour effectuer une action précise qu'il ne peut pas faire seul.....	5
Création du Tool	5
Ajout du Tool à l'agent	6
Agent sans Web Search Tool	6
Agent avec Web Search Tool.....	6
Sortie du Tool	6
Sortie de l'agent	7
Agent sans mémoire.....	7
Agent avec mémoire	8
TP : Chef personnel	8

Création d'un nouveau agent sans System Message

```
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from langchain_ollama import ChatOllama

# Initialiser le modèle Ollama
model = ChatOllama(
    model="llama3.2:3b", # ou mistral, gemma, etc.
    temperature=0
)
agent = create_agent(model=model)

question = HumanMessage(content="Quelle est la capitale de la lune ?")

response = agent.invoke(
    {"messages": [question]}
)

print(response['messages'][-1].content)
```

Agent avec System Message : un message de contrôle qui définit le comportement global du modèle

```
system_prompt = "Vous êtes un auteur de science-fiction ; créez une capitale à la demande des utilisateurs."

scifi_agent = create_agent(
    model=model,
    system_prompt=system_prompt
)

response = scifi_agent.invoke(
    {"messages": [question]}
)

print(response['messages'][-1].content)
```

Agent avec Few-shot learning : une méthode où le modèle apprend une nouvelle tâche ou classe à partir de quelques exemples seulement

```
system_prompt = """
```

```
Vous êtes un auteur de science-fiction et vous devez créer une capitale spatiale à la demande d'un utilisateur.
```

```
Utilisateur : Quelle est la capitale de Mars ?
```

```
Auteur : Marsialis
```

```
Utilisateur : Quelle est la capitale de Vénus ?
```

```
Auteur : Venusovia
```

```
"""
```

```
scifi_agent = create_agent(  
    model=model,  
    system_prompt=system_prompt  
)
```

```
response = scifi_agent.invoke(  
    {"messages": [question]}  
)
```

```
print(response['messages'][-1].content)
```

Agent avec réponse structurée: une sortie organisée selon un format prédéfini, plutôt que du texte libre.

```
system_prompt = """
```

```
Vous êtes un auteur de science-fiction et vous devez créer une capitale spatiale à la demande d'un utilisateur.
```

```
Veillez respecter la structure ci-dessous.
```

```
Nom : Nom de la capitale
```

```
Localisation : Lieu où elle est située
```

```
Ambiance : Description en 2 ou 3 mots
```

```
Économie : Principaux secteurs d'activité
```

```

"""

scifi_agent = create_agent(
    model=model,
    system_prompt=system_prompt
)

response = scifi_agent.invoke(
    {"messages": [question]}
)

print(response['messages'][-1].content)

```

Agent avec réponse structurée en utilisant BaseModel : rendre la réponse facile à exploiter automatiquement par un programme ou un système.

```

from langchain.agents import create_agent
from langchain.messages import HumanMessage
from pydantic import BaseModel

```

```

class CapitalInfo(BaseModel):
    nom: str
    Localisation: str
    Ambiance: str
    Économie: str

```

```

system_prompt = """

```

Vous êtes un auteur de science-fiction et vous devez créer une capitale spatiale à la demande d'un utilisateur.

Veillez respecter la structure ci-dessous.

Nom : Nom de la capitale

Localisation : Lieu où elle est située

Ambiance : Description en 2 ou 3 mots

Économie : Principaux secteurs d'activité

```

"""

agent = create_agent(
    model=model,
    system_prompt=system_prompt,

```

```

        response_format=CapitalInfo
    )

question = HumanMessage(content="Quelle est la capitale de la lune ?")

response = agent.invoke(
    {"messages": [question]}
)

response["structured_response"]

```

Les tools (outils) : une fonction externe que le modèle peut appeler pour effectuer une action précise qu'il ne peut pas faire seul

Création du Tool

```

from langchain.tools import tool

@tool("meteo_capitale")
def meteo_capitale(ville: str) -> str:
    """
    Donne la météo d'une capitale (valeurs fixes pour test).

    Args:
        ville: nom de la capitale
    """
    print("tool meteo_capitale utilisé")

    temperature = 25
    humidite = 60
    pression = 1013

    return (
        f"Météo à {ville} : "
        f"Température = {temperature}°C, "
        f"Humidité = {humidite}%, "
        f"Pression = {pression} hPa"
    )

```

Ajout du Tool à l'agent

```
system_prompt = "Utilises les tools pour répondre aux questions"
agent = create_agent(
    model=model,
    tools=[meteo_capitale],
    system_prompt=system_prompt,
)

question = HumanMessage(content="Quelle est la météo à Capitole lunaire ?")

response = agent.invoke(
    {"messages": [question]}
)

print(response['messages'][-1].content)
```

Agent sans Web Search Tool

```
agent = create_agent(
    model=model,
)
question = HumanMessage(content="Vos connaissances en matière d'apprentissage
sont-elles à jour ?")

response = agent.invoke(
    {"messages": [question]}
)
print(response['messages'][-1].content)
```

Agent avec Web Search Tool

Sortie du Tool

```
from typing import Dict, Any
from tavily import TavilyClient
from dotenv import load_dotenv

load_dotenv()

tavily_client = TavilyClient()

@tool
```

```
def web_search(query: str) -> Dict[str, Any]:  
    return tavily_client.search(query)  
  
web_search.invoke("Qui est le Président de commune actuel de Marrakech ?")
```

Sortie de l'agent

```
agent = create_agent(  
    model=model,  
    tools=[web_search]  
)  
  
question = HumanMessage(content="Qui est le Président de commune actuel de  
Marrakech ?")  
  
response = agent.invoke(  
    {"messages": [question]})  
print(response['messages'][-1].content)
```

Agent sans mémoire

```
agent = create_agent(  
    model= model)  
question = HumanMessage(content="Bonjour, mon nom est Sami et je suis un déve-  
loppeur.")  
  
response = agent.invoke(  
    {"messages": [question]})  
)  
print(response['messages'][-1].content)  
  
question = HumanMessage(content="Quel est mon métier ?")  
  
response = agent.invoke(  
    {"messages": [question]})  
)  
  
print(response['messages'][-1].content)
```

Agent avec mémoire

```
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    model=model,
    checkpointer=InMemorySaver(),
)
question = HumanMessage(content="Bonjour, mon nom est Sami et je suis un développeur.")
config = {"configurable": {"thread_id": "1"}}

response = agent.invoke(
    {"messages": [question]},
    config,
)

question = HumanMessage(content="Quel est mon métier ?")

response = agent.invoke(
    {"messages": [question]},
    config,
)
print(response['messages'][-1].content)
```

TP : Chef personnel

En vous appuyant sur un system message, l'outil de recherche web et le mécanisme de mémoire, vous devez concevoir un agent intelligent jouant le rôle d'un chef cuisinier personnel.

L'agent doit être capable de :

- recevoir la liste des ingrédients disponibles dans le réfrigérateur,
- mémoriser les préférences ou informations fournies par l'utilisateur (mémoire),
- utiliser un outil de recherche web pour compléter ses connaissances culinaires si nécessaire (recettes, techniques, associations d'ingrédients),
- proposer un ou plusieurs plats adaptés aux ingrédients disponibles.